Clean Code - COMPLETE Ordered Notes (C → E → F → G → J → N → T)

C1: Inappropriate Information

Do not store metadata (author, history) inside code comments.

C2: Obsolete Comment

Remove outdated comments immediately.

C3: Redundant Comment

```
# BAD
i += 1  # increment i

# GOOD
index += 1
```

C4: Poorly Written Comment

Write clear, concise, grammatically correct comments.

C5: Commented-Out Code

```
# BAD
# old logic here (never delete)

# GOOD
# DELETE IT (use git history instead)
```

E1: Build Should Be One Step

Project should build with one command.

E2: Tests Should Be One Step

Run all tests with a single command.

F1: Too Many Arguments

```
# BAD
```

```
def create(a,b,c,d,e,f): pass

# GOOD
class Data: pass
def create(data): pass
```

F2: Output Arguments

Avoid modifying arguments. Return values instead.

F3: Flag Arguments

```
# BAD
def process(is_admin):
    if is_admin: ...

# GOOD
def process_admin(): ...
def process_user(): ...
```

F4: Dead Function

Delete unused functions.

G1: Multiple Languages in One File

Avoid mixing HTML, JS, SQL in one file.

G2: Obvious Behavior Missing

Implement expected behavior (e.g., case-insensitive).

G3: Boundary Conditions

Always test edge cases.

G4: Overridden Safeties

Do not disable warnings/tests.

G5: Duplication (DRY)

```
# BAD
def a(): return x+y
def b(): return x+y
```

```
# GOOD
def add(x,y): return x+y
```

G6: Wrong Abstraction Level

Separate high-level vs low-level logic.

G7: Base Class Depends on Child

Base classes must not depend on derived classes.

G8: Too Much Information

Keep interfaces small.

G9: Dead Code

Delete unused code.

G10: Vertical Separation

Keep related code close.

G11: Inconsistency

Follow consistent naming/style.

G12: Clutter

Remove unused variables/functions.

G13: Artificial Coupling

Avoid unnecessary dependencies.

G14: Feature Envy

Move logic to the class that owns the data.

G15: Selector Arguments

```
# BAD
def pay(type):
    if type == "overtime": ...

# GOOD
def pay_overtime(): ...
```

G16: Obscured Intent

```
# BAD
def m(): return i*w

# GOOD
def calculate_salary(hours, rate): return hours*rate
```

G17: Misplaced Responsibility

Put logic where it logically belongs.

G18: Inappropriate Static

Prefer instance methods if polymorphism is needed.

G19: Explanatory Variables

```
total = base + tax + shipping
```

G20: Function Names Should Be Clear

```
# BAD
add(5)

# GOOD
add_days(5)
```

G21: Understand Algorithm

Refactor until logic is clear.

G22: Make Dependencies Explicit

Do not hide dependencies.

G23: Prefer Polymorphism

```
# BAD
if type=="dog": bark()

# GOOD
class Dog: def sound(): return "bark"
```

G24: Follow Conventions

Use standard coding conventions.

G25: Magic Numbers

```
# BAD
if x > 86400

# GOOD
SECONDS_PER_DAY = 86400
```

G26: Be Precise

Handle null, concurrency, edge cases.

G27: Structure Over Convention

Use design to enforce rules.

G28: Encapsulate Conditionals

```
# GOOD
if timer.should_stop():
```

G29: Avoid Negative Conditionals

```
# GOOD
if is_active:
```

G30: Do One Thing

```
# BAD
def process(): validate(); save(); send()
```

G31: Hidden Temporal Coupling

Make order explicit via arguments.

G32: Don't Be Arbitrary

Have consistent structure.

G33: Encapsulate Boundaries

Avoid scattered +1 / -1.

G34: Single Level of Abstraction

Do not mix levels.

G35: Config at High Level

Expose configs at top level.

G36: Law of Demeter

```
# BAD
a.getB().getC().do()

# GOOD
a.do()
```

J1: Use Wildcard Imports

Avoid long import lists.

J2: Don't Inherit Constants

Use static import instead.

J3: Use Enums Instead of Constants

Enums are safer and clearer.

N1: Descriptive Names

Names = 90% readability.

N2: Proper Abstraction Naming

Avoid low-level naming.

N3: Standard Naming

Follow conventions (toString, etc).

N4: Unambiguous Names

Avoid vague names.

N5: Scope-Based Length

Long scope = longer name.

N6: Avoid Encoding

No Hungarian notation.

N7: Describe Side Effects

Function names must reveal behavior.

T1: Insufficient Tests

Test everything.

T2: Use Coverage Tools

Find untested code.

T3: Don't Skip Small Tests

Even trivial tests matter.

T4: Ignored Tests

Indicate ambiguity.

T5: Test Boundaries

Edge cases are critical.

T6: Test Near Bugs

Bugs cluster.

T7: Patterns Reveal Bugs

Analyze failure patterns.

T8: Coverage Patterns

Check executed vs not.

T9: Tests Must Be Fast

Slow tests get ignored.

Conclusion

Clean Code = discipline, clarity, and maintainability.